

StaySpot - Project Requirements & Prompt

1. System Prerequisites

To run the project, your system must have the following installed:

- **Node.js:** Version 16 or above (includes npm)
- **Java:** Version 17 or above
- **Maven:** Version 3.6 or above
- **MySQL:** Version 8.0 or above

2. Technology Stack

The project is built on and requires the following technologies:

Frontend

- **React 19:** Core UI framework
- **Vite:** Build tool and local development server
- **React Router v7:** For handling client-side page navigation (`react-router-dom`)
- **CSS3 & JavaScript ES6+:** Custom CSS3 focusing on a "Glass-morphism" aesthetic (translucent backgrounds, blur effects, clean modern layouts).

Backend

- **Spring Boot 3.3.0 (Java 17):** The main web framework
- **Spring Data JPA:** For database ORM (Object-Relational Mapping)
- **Spring Security:** For handling user authentication
- **JWT (jjwt 0.12.3):** For token-based secure authentication
- **Lombok:** To reduce boilerplate Java code
- **MySQL 8.x:** Relational database

3. Core Functional Requirements

The main features that the system implements:

- **Authentication:** Secure user registration and login with JWT tokens, and BCrypt for password encryption. Token persistence in browser `localStorage` .
- **User Interface:** A modern, clean UI featuring glass-morphism design, fully responsive on both desktop and mobile devices. Real-time form validation.
- **Architecture:** RESTful API connecting a React frontend with a Spring Boot backend, storing user data securely in MySQL database.

Recreation Prompt

The following prompt can be used to recreate the exact project structure and configuration using an AI Coding Assistant:

Please build a full-stack Accommodation Rental Platform named "StaySpot" with a modern glass-morphism UI. The project consists of a React frontend and a Spring Boot backend.

Here are the exact requirements, technical stack, and architecture that you must follow strictly:

1. Technology Stack

Frontend:

- Framework: React 19 with Vite (JavaScript ES6+)
- Routing: React Router v7 (`react-router-dom`)
- Icons: `react-icons`
- Maps: `@react-google-maps/api`
- Styling: Custom CSS3 focusing on a "Glass-morphism" aesthetic (translucent backgrounds, blur effects, clean modern layouts). Do not use Tailwind unless necessary.
- State & Auth: JWT tokens persisted in browser `localStorage` .

Backend:

- Framework: Spring Boot 3.3.0 (Java 17)
- Build Tool: Maven
- Database: MySQL 8.x
- ORM: Spring Data JPA
- Security: Spring Security with BCrypt password encryption
- Authentication: JWT using `jjwt` (version 0.12.3)
- Boilerplate Reduction: Lombok
- Validation: Spring Boot Starter Validation

2. Core Features to Implement

Authentication System

- User Registration with fields: `username` , `email` , `password` , `fullName` .
- User Login with `username` and `password` .
- Passwords must be hashed using BCrypt before saving to the database.
- Secure API endpoints using Spring Security and JWT.
- Handle CORS to allow frontend (`http://localhost:5173`) to communicate with backend (`http://localhost:8080`).

Frontend Pages

- **Landing Page:** A beautiful hero section introducing the platform.
- **Auth Pages:** Login and Register forms with real-time validation.
- **Dashboard:** A protected route showing user details (accessible only if a valid JWT exists in `localStorage`).
- **Landlord Interface (Mockup/Views):** Landlord Form, Landlord Approval, and Landlord Status pages.

3. Database Schema

Create the following table structure using JPA Entities (or provide a `MYSQL_SETUP.sql` initialization script):

```
CREATE TABLE users (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  username VARCHAR(50) UNIQUE NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  password VARCHAR(255) NOT NULL,  
  full_name VARCHAR(100) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

4. API Endpoints

Expose the following REST APIs in the backend:

1. POST /api/auth/register (Accepts username, email, password, fullName. Returns JWT token and success status)
2. POST /api/auth/login (Accepts username, password. Returns JWT token and success status)
3. GET /api/health (Returns "Backend is running!" - Unsecured endpoint for testing)

5. Expected Directory Structure

Please generate the code following this exact monolithic repository structure:

```
StaySpot/
├── backend/
│   ├── src/main/java/com/stayspot/
│   │   ├── controller/ (AuthController, HealthController)
│   │   ├── service/ (UserService)
│   │   ├── repository/ (UserRepository)
│   │   ├── model/ (User Entity)
│   │   ├── dto/ (RegisterRequest, LoginRequest, AuthResponse)
│   │   ├── security/ (JwtUtil, PasswordEncoderConfig, SecurityFilterChain config)
│   │   └── StaySpotApplication.java
│   ├── src/main/resources/application.properties
│   └── pom.xml
├── frontend/
│   ├── src/
│   │   ├── pages/ (Landing.jsx, Register.jsx, Login.jsx, Dashboard.jsx, etc.)
│   │   ├── services/ (api.js - Axios/Fetch configuration with JWT interceptors)
│   │   ├── App.jsx (React Router configuration)
│   │   ├── main.jsx
│   │   └── index.css (Glass-morphism CSS variables and utilities)
│   ├── package.json
│   └── vite.config.js
```

Execution Steps

1. Please start by writing the Backend code (Entities, JWT Security Config, AuthController, application.properties, and pom.xml).
2. Once the backend is complete, write the Frontend code (Vite setup, React Router, Auth Context/API service, and the Glass-morphism UI pages).
3. Ensure the CSS makes the app look premium, responsive, and adheres to the glass-morphism aesthetic.